Node.js & TypeScript Project Setup and Vercel Deployment Guide

This document provides a comprehensive step-by-step guide for setting up, configuring, building, and deploying a modern Node.js application built with TypeScript, Bun, Prisma, TSUP, and Vercel.

1. Project Initialization & Setup

Step 1.1: Clone the Repository

Clone your existing Git repository to your local workspace:

```
git clone <repository-url>
cd <repository-name>
```

Step 1.2: Install Dependencies

Install all project dependencies using the bun package manager:

```
bun install
```

Step 1.3: Environment Configuration

Create a .env file in the root directory and configure the necessary environment variables required for your application:

```
PORT=5000
DATABASE_URL="postgresql://user:password@localhost:5432/mydb?schema=public"
REDIS_URL="redis://default:password@localhost:6379"
GOOGLE_APP_PASSWORD="your-google-app-password"
```

2. Database Setup (Prisma)

Generate the Prisma Client and apply database migrations using Bun:

Step 2.1: Generate Prisma Client

```
bunx prisma generate
```

Step 2.2: Apply Migrations

```
bunx prisma migrate dev --name init
```

3. TSUP Build Configuration

TSUP is a zero-config TypeScript bundler powered by esbuild. Install it as a dev dependency and create a dedicated configuration file.

Step 3.1: Install TSUP

```
bun add -D tsup
```

Step 3.2: Create tsup.config.ts

Create a tsup.config.ts file in your project root with the following setup, including a CJS require shim for ESM compatibility:

```
import { defineConfig } from "tsup";

export default defineConfig({
  entry: ["src/server.ts"],
  format: ["esm", "cjs"],
  target: "esnext",
  outDir: "dist",
  clean: true,
  bundle: true,
  splitting: false,
  sourcemap: true,
  // Add banner to shim require() for CJS dependencies in ESM context
  banner: {
    js: `
      import { createRequire } from 'module';
      const require = createRequire(import.meta.url);
    `,
  },
});
```

4. Package Scripts Configuration

Update your package.json file to include development, build, and production start scripts:

```
{
  "scripts": {
    "dev": "tsx watch src/server.ts",
    "build": "tsup",
    "start": "node dist/server.js"
```

```
  }
}
```

5. Vercel Deployment Configuration

Configure Vercel serverless deployment using a custom vercel.json file located in the root directory.

Step 5.1: Create vercel.json

```
{
  "version": 2,
  "builds": [
    {
      "src": "dist/server.js",
      "use": "@vercel/node"
    }
  ],
  "routes": [
    {
      "src": "/(.*)",
      "dest": "dist/server.js"
    }
  ]
}
```

Step 5.2: Deploy via Vercel CLI

Run the following commands in your terminal to deploy your application to production:

```
npm i -g vercel
vercel login
vercel --prod
```

Step 5.2: Manual ENV Setup

6. Serverless Limitations & Features to Avoid

When deploying serverless Node.js applications to free-tier cloud platforms like Vercel, server processes are stateless and short-lived. Avoid the following architecture patterns:

Feature / Pattern	Why Avoid in Serverless?	Recommended Alternative
Cron Jobs (Node-cron / setInterval)	Serverless instances spin down when inactive, halting background timers.	Use Vercel Cron Jobs, Upstash QStash, or GitHub Actions.
WebSockets / Realtime (Socket.io)	Persistent TCP connections are unsupported on serverless functions.	Use Pusher, Ably, Supabase Realtime, or Server-Sent Events (SSE).
Direct File Uploads / File System (fs)	The local disk is read-only and ephemeral; saved files vanish across requests.	Upload directly to AWS S3, Cloudinary, or Supabase Storage via signed URLs.
In-Memory Caching / State	Memory is not shared across function instances or persistent across requests.	Use external state stores like Redis (Upstash) or a database.